Clase 1, Parte 2: Repaso Práctico de Python

Duración Estimada: 90 minutos

En esta segunda parte, vamos a codificar. Reforzaremos los conceptos de Python que son fundamentales para el desarrollo de APIs: las estructuras de datos y la manipulación de archivos.

2.1 Manipulación de Datos (60 minutos)

Objetivo: Reforzar el uso de diccionarios y listas, y practicar la lógica con bucles y condicionales.

Paso 1: Crear el Archivo Principal

En VS Code, asegúrense de estar en la carpeta python_backend_tpi y de que su entorno virtual esté activo. Para crear un nuevo archivo, pueden ir al menú File > New File... y guardarlo con el nombre app.py.

Paso 2: Escribir el Código

Dentro de app.py, definan una lista de diccionarios que represente una base de datos de productos.

```
productos = [
  {"nombre": "Laptop", "precio": 1200, "stock": 15},
  {"nombre": "Mouse", "precio": 25, "stock": 5},
  {"nombre": "Teclado", "precio": 75, "stock": 25},
  {"nombre": "Monitor", "precio": 300, "stock": 8},
]
```

Bucle for y Lógica Condicional:

```
productos_bajo_stock = []
for producto in productos:
  print(f"Producto: {producto['nombre']}, Precio: ${producto['precio']}")
  if producto['stock'] < 10:
    productos_bajo_stock.append(producto)

print("\nProductos con bajo stock:")
print(productos_bajo_stock)
```

Función:

```
def calcular_promedio_precio(lista):
    if not lista:
        return 0
    total_precio = sum(p['precio'] for p in lista)
    return total_precio / len(lista)

precio_promedio = calcular_promedio_precio(productos)
print(f"\nEl precio promedio de los productos es: ${precio_promedio:.2f}")
```

Paso 3: Ejecutar el Código

En la terminal de VS Code, ejecuten python app.py. Verifiquen la salida en consola.

2.2 Manejo de Archivos (30 minutos)

Objetivo: Practicar la lectura de archivos CSV y la conversión a estructuras de datos de Python.

Paso 1: Crear el Archivo de Datos

Creen un nuevo archivo en la misma carpeta del proyecto y guárdenlo con el nombre datos.csv. Agreguen el siguiente contenido:

```
id,nombre,precio,stock
1,Laptop,1200,15
2,Mouse,25,5
3,Teclado,75,25
4,Monitor,300,8
```

Paso 2: Leer el Archivo CSV (Ampliación Técnica y Buenas Prácticas)

Modifiquen app.py para que ahora lea este archivo. A nivel universitario y profesional, no basta con "leer" el archivo; debemos hacerlo de forma segura y consistente. Para ello:

1. **Uso de Context Managers (with)**: Para leer el archivo, utilizaremos un "Context Manager" (el bloque with open(...)). Esta es una buena práctica estándar en la industria porque asegura que el archivo se cierre automáticamente, liberando recursos en el sistema operativo (ya sea que utilicen Windows o Linux), incluso si el programa falla a la mitad de la lectura por un error inesperado.
2. **Mapeo a Diccionarios (csv.DictReader)**: Usaremos la librería estándar csv de Python. La

función csv.DictReader() es extremadamente útil porque lee automáticamente la primera fila del CSV (las cabeceras) y convierte cada fila subsiguiente en un diccionario donde las claves son dichas cabeceras.

3. **Casteo de Tipos (Conversión Crítica)**: ¡Atención a este detalle técnico! Al leer un archivo de texto plano como lo es un CSV, Python asume por defecto que todos los datos son texto (string). Si queremos operar matemáticamente después, un precio leído como "1200" (texto) causará un error fatal. Debemos convertirlos explícitamente a enteros (int()) apenas los extraemos.

Con la ayuda del docente, agreguen esta lógica al final de su archivo app.py:

```
import csv

# Lista donde guardaremos los datos provenientes del archivo
productos_desde_csv = []

# 1. Uso del Context Manager 'with' en modo lectura ('r') y codificación estándar
with open('datos.csv', mode='r', encoding='utf-8') as archivo_csv:
    # 2. DictReader convierte cada fila automáticamente en un diccionario
    lector_diccionario = csv.DictReader(archivo_csv)

    for fila in lector_diccionario:
        # 3. Casteo de Tipos: Los datos llegan como strings, los forzamos a ser enteros (int)
        fila['id'] = int(fila['id'])
        fila['precio'] = int(fila['precio'])
        fila['stock'] = int(fila['stock'])

        productos_desde_csv.append(fila)

print("\nLista de productos desde el archivo CSV:")
print(productos_desde_csv)
```

Paso 3: Verificar los Resultados

Ejecuten python app.py. Impriman la lista de productos que obtuvieron del archivo CSV. Verán que es idéntica a la que creamos manualmente en el punto anterior, pero ahora con los tipos de datos numéricos correctos y seguros.